

Authorization

Who can see what, and where generated code leaves the check out

A missing authorization check produces no error and no visible symptom, so the app keeps working until somebody requests data belonging to another user. That makes it difficult to catch while reading a diff, and it is among the more common serious defects in AI-assisted code.

Format	Self-study lab manual. Five sittings, roughly five hours.
You need	A free Supabase project, a browser, an AI coding assistant.
Assumes	Roughly what a table is, roughly what a request is. No programming experience — all SQL is provided or pattern-matched.
Method	Write down what you expect, run the code, compare.

Contents

Front matter — How to use this	3
Setup — Twenty minutes, before Sitting One	4
Sitting One — The trust boundary	5
Lab 1 — What the browser can actually do	5
Reading A — Authentication is not authorization	7
Sitting Two — Reading for the gap	9
Lab 2 — Find the check	9
Reading B — What a policy actually does	11
Sitting Three — Writing policies	13
Lab 3 — Close the hole you opened	13
Lab 4 — Write the policies	14
Sitting Four — Auditing	17
Reading C — Seven ways authorization goes missing	17
Lab 5 — Audit code your assistant writes	18
Sitting Five — Your own project	20
Reference card — The checklist and the house rules	20
Lab 6 — Your own project	21
Troubleshooting — When you are stuck	22
Reference 1 — Schema	23
Reference 2 — Seed	23
Reference 3 — The impersonation harness	24
Reference 4 — Lab 4 assertions	25
Further — If you want to go deeper	26

FRONT MATTER

How to use this

Every experiment in this manual has the same three steps: write down what you expect, run the code, then compare. It costs a couple of minutes each time, and it's where most of the learning happens — a wrong prediction points at the specific part of your model that needs fixing, which reading rarely surfaces on its own.

Specificity helps here. 'Probably an error' leaves nothing much to correct, while 'zero rows, because the policy filters on `created_by`' either holds up or shows you exactly what to revise.

The calibration log

Keep a file called `calibration.md` open while you work, with a row for each experiment.

#	I predicted	What happened	What I got wrong	Rule I'm updating
1.1				
1.2				

Fill in the prediction before you run anything. Once you've seen the result it gets hard to recover what you actually believed beforehand, and the comparison loses most of its value. 'No idea' is a usable entry.

If the fourth column keeps coming up empty, the predictions were probably too broad to check against anything, so it's worth pushing for more detail in the next few.

Answers

Everything here is answered somewhere in the document, and you can read ahead whenever you want. Rewriting the answer from a blank file afterwards is what makes it stick.

Lab 4 works a little differently. Its key is a set of SQL assertions that run against your own database and either pass or fail, so you can check yourself there either way.

Sittings

Sitting	Contents	Time
One	Lab 1 · Reading A	50 min
Two	Lab 2 · Reading B	60 min
Three	Lab 3 · Lab 4	70 min
Four	Reading C · Lab 5	60 min
Five	Lab 6 · The reference card	45 min

Stop at the boundaries where you can. The gap between sittings three and four earns its place — Lab 5 builds directly on the mistakes from Lab 4, and that material takes a little time to settle.

SETUP

Twenty minutes, before Sitting One

1. Create a free Supabase project. From **Settings** → **API** note three things in a scratch file: the **project URL**, the **anon key**, and the **service_role key**.
2. In **Authentication** → **Users** create three users manually. Any password. Copy each user's **UUID**, which you'll need throughout, and label them A, B and C in your notes.
3. In the SQL editor, run the schema from Reference 1.
4. Run the seed from Reference 2, substituting your three UUIDs where marked.
5. Verify: `select count(*) from documents;` returns 5.

The schema is a small collaborative documents app: workspaces, memberships, documents, comments. Users A and B share workspace 1. User B is alone in workspace 2 and is its admin. User C is in neither. Every lab depends on that asymmetry.

NOTE THE STARTING STATE

Row level security is deliberately **off**, which is also the default state of every table you create in Supabase. The first experiments therefore run against tables with no protection on them.

The trust boundary

Where you are standing relative to the data, and what a browser can do with the key that every visitor to the app receives.

LAB 1

What the browser can actually do

Four experiments, each with a prediction to write down first.

Experiment 1.1 Open a browser tab, open the devtools console, and run:

```
const URL = "https://YOUR-PROJECT.supabase.co";
const ANON = "YOUR_ANON_KEY";
const r = await fetch(`${URL}/rest/v1/documents?select=*`, {
  headers: { apikey: ANON, Authorization: `Bearer ${ANON}` }
});
console.table(await r.json());
```

PREDICT

How many rows come back, and whose? You aren't logged in as anybody, and you hold only the key that every visitor to a Supabase app receives.

Experiment 1.2 Pick a document belonging to user C. Then:

```
await fetch(`${URL}/rest/v1/documents?id=eq.THAT_DOCUMENT_ID`, {
  method: "DELETE",
  headers: { apikey: ANON, Authorization: `Bearer ${ANON}` }
});
```

PREDICT

Does it delete? Re-run 1.1 to check.

Experiment 1.3 Still anonymous:

```
await fetch(`${URL}/rest/v1/memberships`, {
  method: "POST",
  headers: { apikey: ANON, Authorization: `Bearer ${ANON}`,
    "Content-Type": "application/json" },
  body: JSON.stringify({ workspace_id: "WORKSPACE_2_ID",
    user_id: "USER_C_UUID", role: "admin" })
});
```

PREDICT

Can an anonymous stranger make user C an admin of a workspace they were never invited to?

Experiment 1.4 Now a conceptual one. Suppose the app's frontend contains:

```
{user.role === 'admin' && <DeleteButton onClick={deleteDoc} />}
```

PREDICT

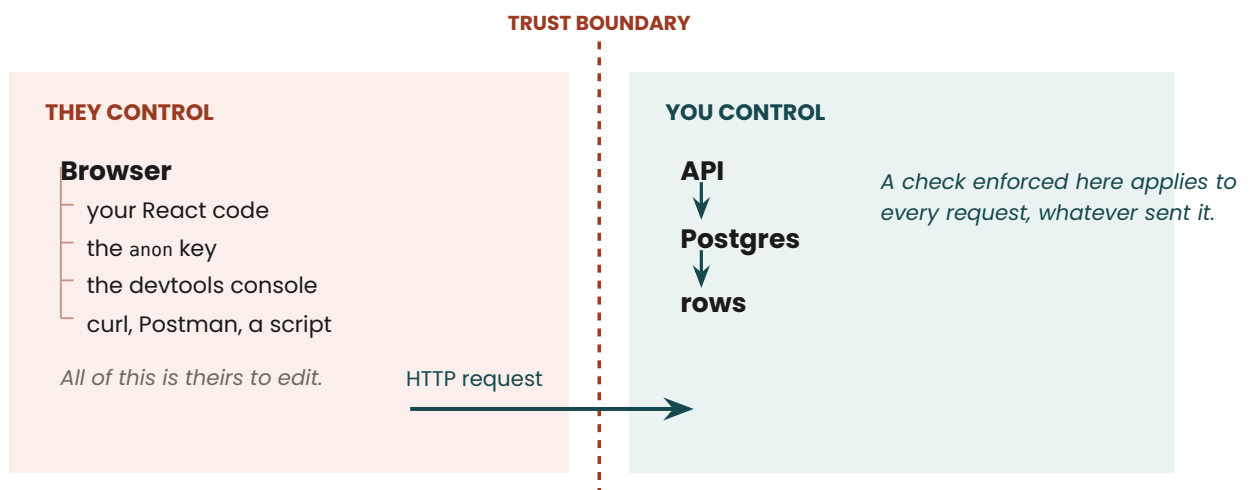
Does that line change the outcome of Experiment 1.2? Write your answer before reading on.

What the experiments show

Those were ordinary HTTP requests to a public endpoint, sent with the key the app publishes to every visitor, and the database answered them. The anon key is designed to sit in the browser, since that's how the frontend talks to the database at all.

Which sets up the distinction the rest of the module runs on. A check written in browser code can always be skipped, because the attacker sends the request your component would have sent with the parts they don't want removed. A check enforced by the database applies to every request that arrives, whatever sent it.

So finding it in devtools is expected rather than a sign that something leaked.



Copying that diagram out by hand is a quick way to test whether the split is clear to

you, since it’s the one distinction the later labs assume.

For Experiment 1.4: the JSX line has no effect on the outcome of 1.2, because the attacker never renders your components. Frontend conditionals serve honest users who are working through the interface.

READING A

Authentication is not authorization

Two separate questions, often handled as one under the heading of ‘permissions’.

- **Authentication:** who are you? Resolved once, at login, and it produces an identity.
- **Authorization:** are you allowed to do this specific thing to this specific row? Resolved on every operation.

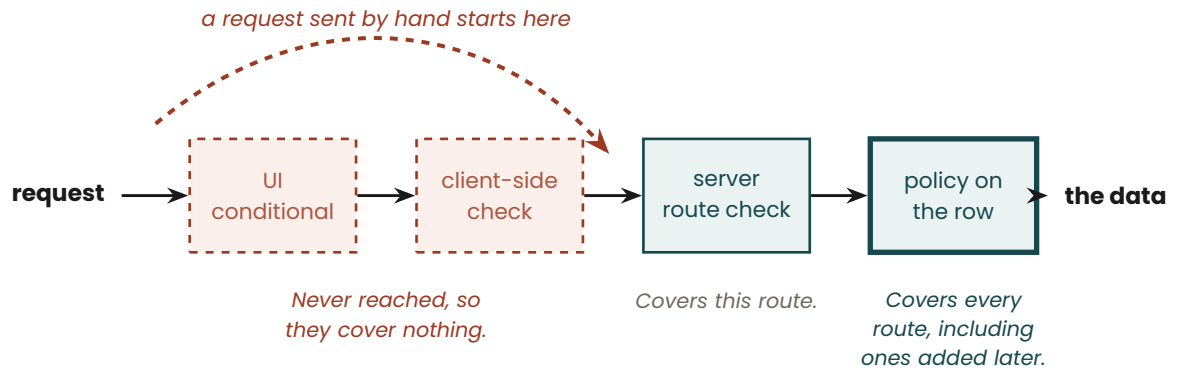
Login can work correctly while authorization is entirely absent, which is the state your lab project is in right now. The confusion worth naming is that **‘there is a logged-in user’ is a different claim from ‘this row belongs to them’** — and code that checks the first while skipping the second is the most common authorization defect in generated code, since it’s shorter to write and there is a visible check for a reviewer to land on.

The four places a check can live

Where	Real?	What it does
UI conditional hide the button	FAKE	Helps usability, since it stops honest users hitting errors.
Client-side code if (user.isAdmin)	FAKE	Nothing, because the attacker doesn’t run your code.
Server / API route check in the handler	REAL	Covers the requests that go through this route.
Database a policy on the row	REAL	Covers every request that reaches the data.

The first two are worth having, since a UI that shows buttons which always fail is confusing to use. They just don’t survive a request that skips the UI.

The gap between the last two is narrower and matters more. A check in a route handler covers that route, so it won’t apply to a second route added next month that touches the same table, or to a background job, or to an endpoint your assistant adds while you skim the diff. A check in the database covers paths that don’t exist yet, which is why it’s usually the one to write first.



Why generated code lands on the first two

- The UI conditional is what ‘add permissions to this’ looks like in the code already on screen.
- The database policy lives in a different system — a migration file or a dashboard — usually outside the files the model was asked to change.
- Client-side checks are shorter, and they read as complete.
- A request to make something work can be satisfied by removing the check that blocks it.

LOG ENTRY FOR SITTING ONE

Add a line to `calibration.md`: which of the four locations you would have assumed was sufficient before today.

Reading for the gap

Five snippets to judge, then the mechanism that explains why the judgements come out the way they do. This is the sitting where the material stops being specific to Supabase.

LAB 2

Find the check

For each snippet, write down where authorization happens, whether it holds, and a one-line fix. Commit to all five before reading any answer. Each one is a shape that generated code produces routinely.

A

```
{user.role === 'admin' && (
  <button onClick={() => supabase.from('invoices').delete().eq('id',
    invoice.id)}>
    Delete invoice
  </button>
)}
```

B

```
export async function getDocument(id) {
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error('Not logged in');
  return supabase.from('documents').select('*').eq('id', id).single();
}
```

C

```
// app/api/notes/route.js -- runs on the server
export async function POST(req) {
```

```
const { userId, content } = await req.json();
return db.insert('notes', { user_id: userId, content });
}
```

D

```
create policy "members can read documents" on documents
for select using (
  workspace_id in (select workspace_id from memberships where user_id =
    auth.uid())
);
```

...where memberships has RLS enabled, with the policy using (true).

E

```
// lib/admin.js
import { createClient } from '@supabase/supabase-js';
export const db = createClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL,
  process.env.NEXT_PUBLIC_SUPABASE_SERVICE_KEY
);
```

ANSWERS

A — The role check is in JSX, so the `delete()` call can be issued without rendering the button, as you did in Experiment 1.2. *Fix:* a delete policy on invoices restricted to admins of the owning organisation.

B — Authentication only. It confirms somebody is logged in, then fetches any document by id, so changing the id in the URL returns another user's document. *Fix:* filter by ownership in the query and add a select policy, since the query filter covers only this function and there will be others.

C — The identity arrives in the request body, so the caller declares who they are and the server accepts it, which lets anyone write notes as anyone. Being server-side code doesn't help here, because the input still came from the client. *Fix:* take the user from the session and ignore any `userId` in the payload.

D — The documents policy is correct, but memberships is world-readable, so every workspace and member can be enumerated. If inserts to memberships are also unrestricted, an attacker can grant themselves membership and the documents policy will then admit them, which is what you did in Experiment 1.3. *Fix:* restrict memberships to rows the caller is party to, and keep client-side writes off a table that grants access.

E — The most severe of the five. `service_role` bypasses every policy by design, and `NEXT_PUBLIC_` compiles the value into the JavaScript bundle sent to every visitor, so the policies elsewhere in the project no longer apply to anyone who reads it out of devtools. *Fix:* remove the prefix, keep the key server-only, and rotate it, since it should be treated as compromised from the

moment it shipped.

Snippet D generalises: a policy that queries another table inherits that table's protection, so authorization has to hold across the whole data graph rather than one table at a time.

READING B

What a policy actually does

A row level security policy is a `WHERE` clause the database adds to every query, and the caller can't remove it because it never appears in their code. A request for all documents against the policy using `(created_by = auth.uid())` runs as ... where `created_by = <the caller>`.

That's also why it holds better than a check in a route handler: it can't be omitted from a future code path, since it isn't in any code path.

Five things to know

1 Enabling RLS and writing a policy are separate steps

```
alter table documents enable row level security;
```

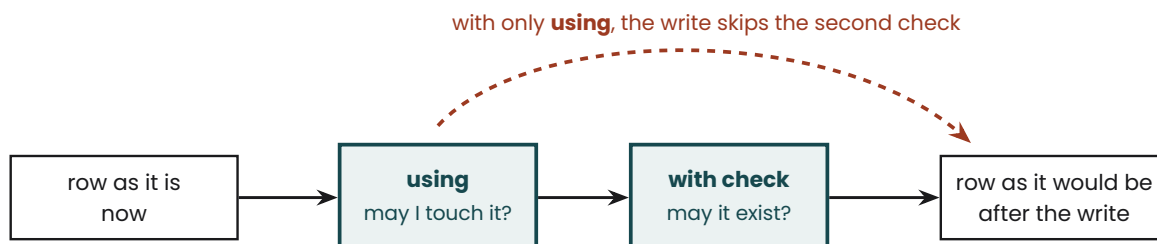
Policies have no effect without that line. With the line and no policies, the table denies everything, which is worth knowing in advance because turning RLS on will break your app before it protects it.

2 Policies are per-operation

`select`, `insert`, `update` and `delete` are separate permissions, so a table with only a `select` policy still accepts writes from anyone.

3 using versus with check

`using` governs which existing rows you may see or touch, while `with check` governs whether a row you are writing is allowed to exist.



The row is examined on the way in, and nothing examines where it lands.

An update needs both clauses, since with only using a user can see their own row and then set `created_by` or `workspace_id` to a value that puts it somewhere they shouldn't be able to reach.

4 `service_role` bypasses every policy

It exists for trusted server-side work. Two rules follow, and Lab 2E shows why: keep it out of anything a browser receives, and don't reach for it to clear a permissions error, since the policy that raised the error is the thing to look at.

5 `auth.uid()` is the caller's verified identity

It comes from a signed token rather than the request body, so it can't be supplied by the caller. Ownership checks should be written against it.

Writing policies

The hands-on core. You close the hole opened in Sitting One, then write a full policy set against an executable grader. Two things are set up to go wrong.

LAB 3

Close the hole you opened

Four short steps, each with a prediction first.

Experiment 3.1 Run `alter table documents enable row level security`; then re-run Experiment 1.1 from the console.

PREDICT

The row count.

Experiment 3.2 Predict what an authenticated user A would now see, then check with the impersonation harness in Reference 3. You'll use it constantly from here, so it's worth getting working now.

Experiment 3.3 Add:

```
create policy "read own documents" on documents
for select using (created_by = auth.uid());
```

PREDICT

What does A see, what does B see, and what does the anonymous console see?
All three, before running any of them.

Experiment 3.4 Re-run Experiment 1.2, the anonymous DELETE.

PREDICT

Does it still delete? You have added only a select policy. Write down your reasoning as well as the answer.

DEBRIEF ON 3.4

The delete is blocked because RLS denies by default and no delete policy permits it, rather than because you restricted deletes. The distinction matters once you add a delete policy: a permissive one will open an operation you currently think is closed.

Also worth noting: no application code changed in this lab. The vulnerability and the fix both sat outside the files the app is written in.

LAB 4

Write the policies

Write policies for the six requirements below, then run the assertions in Reference 4, which print PASS or FAIL per requirement.

SEQUENCING

Reading the solution before the assertions pass makes the rest of the lab less useful, since the failed attempts are where the detail sticks. Forty minutes is a reasonable point to give up and read.

Requirements

- 1** A user can read documents in workspaces they belong to, and no others.
- 2** A user can create a document only in a workspace they belong to, with `created_by` set to themselves.
- 3** A user can edit a document they created, and can't move it to a different workspace.
- 4** A document can be deleted by its creator, or by an admin of its workspace.
- 5** Comments are readable by any member of the document's workspace, and editable only by their author.
- 6** Nobody can read a `memberships` row for a workspace they don't belong to.

Two things are set up to go wrong

infinite recursion detected in policy for relation 'memberships'

This comes up on requirement 1 or 6. The documents policy queries memberships, which has its own policy, whose evaluation queries memberships again, so Postgres stops.

The way out is a function that runs with elevated rights, so its internal query isn't policy-checked:

```
create or replace function is_member(ws uuid)
returns boolean
language sql security definer stable
as $$ select exists (
  select 1 from memberships where workspace_id = ws and user_id = auth.uid()
) $$;
```

Then using `(is_member(workspace_id))`. The function ignores RLS, so its body has to stay narrow and filter on `auth.uid()` itself — a security definer function with a loose body reopens the access the policies were added to restrict.

Requirement 3 looks finished before it is

The common version is using `(created_by = auth.uid())` on its own. To see the gap, act as user B and update your own document to set `workspace_id` to workspace 3, which B doesn't belong to. It succeeds, and the document is now in a workspace B can't read. The same move in the other direction inserts a row into a workspace the caller can't see. Adding with `check` closes both.

SOLUTION

```
-- helper (see the recursion note above)
create or replace function is_member(ws uuid)
returns boolean language sql security definer stable
as $$ select exists (
  select 1 from memberships where workspace_id = ws and user_id =
    auth.uid()
) $$;

-- 1
create policy "read docs in my workspaces" on documents
for select using (is_member(workspace_id));

-- 2
create policy "create docs in my workspaces" on documents
for insert with check (is_member(workspace_id) and created_by =
  auth.uid());

-- 3 both clauses required
create policy "edit own docs" on documents
for update
using (created_by = auth.uid())
with check (created_by = auth.uid() and is_member(workspace_id));
```

```
-- 4
create policy "delete own or as admin" on documents
for delete using (
  created_by = auth.uid()
or exists (
  select 1 from memberships m
  where m.workspace_id = documents.workspace_id
    and m.user_id = auth.uid() and m.role = 'admin'
)
);

-- 5
create policy "read comments in my workspaces" on comments
for select using (
  exists (select 1 from documents d
    where d.id = comments.document_id and
    is_member(d.workspace_id))
);
create policy "edit own comments" on comments
for update using (author_id = auth.uid()) with check (author_id =
  auth.uid());

-- 6
create policy "read own memberships" on memberships
for select using (user_id = auth.uid());
```

Then write the seven policies into a blank file from memory and run the assertions again.

Auditing

Seven shapes to recognise in a diff, then an audit of code your own assistant writes for a feature it hasn't seen before.

READING C

Seven ways authorization goes missing

Each of these is a shape to recognise while reading, with the signal that gives it away.

1 The UI-only gate. The button is hidden and the endpoint is open.

Tell: the only mention of a role or permission in the diff sits inside JSX.

2 Authenticated but not authorized. The session is confirmed and ownership isn't.

Tell: `if (!user) throw` followed by a query whose only filter is `.eq('id', id)`.

3 New table with RLS left off. A migration adds a table, which is then exposed through the API with no protection.

Tell: `create table` in a diff with no accompanying `enable row level security`.

4 `service_role` used to clear an error. Reached for when a permission error blocks progress, or shipped to the client.

Tell: `service` or `admin` in a client-side file, any `NEXT_PUBLIC_*` holding a secret, a commit message about fixing permissions.

5 Partial coverage. Reads restricted and writes open, or using `without` with `check`.

Tell: fewer than four operations addressed on a table users can write to.

6 Identity taken from the request. `user_id` arrives in the payload and becomes the ownership field.

Tell: any `userId` destructured from a request body.

7 The unprotected neighbour. The target table is protected and a join or lookup table isn't, so the data is reachable sideways, or access can be granted by writing to it.

Tell: a policy whose correctness depends on another table, without a check of that table's own policies.

All seven have now come up in the labs. Go back through `calibration.md` and label each entry with the mode it belongs to.

LAB 5

Audit code your assistant writes

Open a fresh session with your coding assistant and give it exactly this:

PREDICT

'Add a feature to my Supabase app: let a user share a document with another user by email. Include the migration, the API route, and the UI component.'

Don't mention security, permissions or RLS, since the point is to see the default rather than to find out whether the model can do better when asked.

Before reading the output, write down which of the seven modes you expect to appear, ranked. Then audit what came back against all seven, marking each as present, absent, or not applicable. Finish by asking whether you'd have merged this a week ago.

What tends to come up

Across many runs of this prompt, modes 3 and 5 are the most frequent: a new shares table appears, often with RLS enabled and a select policy, and often with no delete policy, so anyone can revoke anyone's access. Mode 6 shows up in the API route, since taking `documentId` and `sharedWithUserId` from the request body is the shortest implementation that looks right. Mode 1 appears in most runs and does little harm on its own, though it becomes the only protection when nothing else is present.

Your run will differ, since you're checking your own tool at its current version rather than this snapshot of it.

Two variations

Run 2

The same prompt with the house rules from the next section pasted in. Diffing the two results gives you a measure of what a conventions file buys in your own project.

Run 3

Paste the generated code back and ask what authorization gaps it has. Comparing what it catches unprompted against what it only concedes once you name the mode tells you how much of the review you can hand over.

Your own project

The checklist, the house rules, and an audit of a project with real data in it.

REFERENCE CARD

The checklist and the house rules

Seven questions for a diff that touches data

Work from the diff itself, since a summary describes what the change was meant to do.

- 1 Where does the identity come from — the session, or the request?
- 2 Does the query filter by ownership, or only by id?
- 3 Which key is in use, and is `service_role` server-only?
- 4 If a table is created, is RLS enabled and are all four operations covered?
- 5 Does every write policy have both `using` and `with check`?
- 6 Is a UI conditional the only protection present?
- 7 Can the data be reached through a related table?

The direct test

Log in as user B and request user A's row by id. Thirty seconds with the harness in Reference 3, and it produces evidence rather than an inference, so it's worth running whenever the checklist leaves you unsure.

House rules

Put this in whatever file your assistant reads: `CLAUDE.md`, `AGENTS.md`, `CONTRIBUTING.md`.

```
## Authorization rules - do not deviate without asking
```

- Every new table gets ``enable row level security`` in the same migration that creates it, plus explicit policies for select/insert/update/delete.
- Ownership is always derived from ``auth.uid()``. Never from a request body, query param, or client-supplied field.
- Every ``for update`` policy has both ``using`` and ``with check``.
- ``service_role`` is server-only and is never used to resolve a permissions error. If a query is blocked by a policy, examine the policy - not the key.
- UI conditionals on roles are usability only, never sole protection.
- When adding a data-access path, state in your summary which policy protects it. If none does, say so explicitly rather than proceeding.

The last rule turns an omission into a sentence you can notice while reading, which is easier than inferring it from what the diff doesn't contain.

LAB 6

Your own project

This lab applies the checklist to a project with real data in it, rather than the lab project.

- 1** List every table, and for each one note whether RLS is on and which operations have policies.
- 2** Pick the table whose exposure would matter most.
- 3** Run the direct test: authenticate as one user, request another user's row by id.
- 4** Write down what you found, including a finding that everything was in order.
- 5** Fix the worst one, and verify with an assertion rather than by looking at the app.

IF YOU FIND SOMETHING

If a `service_role` key was ever client-visible, rotate it. If data was exposed, your logs will show whether anyone accessed it, and that question is separate from patching the policy.

IF YOU FIND NOTHING

Record why, specifically. 'Every table has RLS on with four policies each, and cross-user reads fail on the three tables holding user data' is checkable later, in a way that 'it looked fine' isn't.

How to tell whether this worked

Reciting the seven modes isn't much of a signal. A better one is whether, a few weeks from now, you find yourself asking which policy protects a piece of data while reviewing something for an unrelated reason.

TROUBLESHOOTING

When you are stuck

The app returns nothing after enabling RLS

Expected. RLS with no policies denies everything, so the table needs a select policy. Turning RLS back off restores the app and removes the protection.

infinite recursion detected in policy for relation X

The policy on X consults a table whose policy consults X. Use the security definer helper from Lab 4 and keep its body narrow.

new row violates row-level security policy

An insert or update was blocked by with check, or there's no insert policy. Work out what the row would have looked like after the write and whether the policy should permit it, since often it correctly shouldn't and the calling code is what needs changing.

It works in the SQL editor but not from the app

The SQL editor runs as a privileged role that bypasses RLS, so it will show you rows a real caller can't see. Use the harness in Reference 3 instead.

The policy looks right and still denies everything

Check that `auth.uid()` isn't null. Inside the harness, `select auth.uid();` should return your user, and if it returns null the impersonation setup needs fixing before any policy will pass.

Several hours in and confused

Redoing Lab 3 takes about ten minutes and re-establishes the core sequence, which is usually enough to locate what went missing.

Reference

REFERENCE 1

Schema

```
create table workspaces (  
  id uuid primary key default gen_random_uuid(),  
  name text not null,  
  created_at timestamptz default now()  
);  
  
create table memberships (  
  id uuid primary key default gen_random_uuid(),  
  workspace_id uuid not null references workspaces(id) on delete cascade,  
  user_id uuid not null references auth.users(id) on delete cascade,  
  role text not null default 'member' check (role in ('member','admin')),  
  unique (workspace_id, user_id)  
);  
  
create table documents (  
  id uuid primary key default gen_random_uuid(),  
  workspace_id uuid not null references workspaces(id) on delete cascade,  
  created_by uuid not null references auth.users(id),  
  title text not null,  
  body text default ''  
);  
  
create table comments (  
  id uuid primary key default gen_random_uuid(),  
  document_id uuid not null references documents(id) on delete cascade,  
  author_id uuid not null references auth.users(id),  
  body text not null  
);
```

REFERENCE 2

Seed

Replace the three UUIDs with your users A, B and C.

```

insert into workspaces (id, name) values
  ('11111111-1111-1111-1111-111111111111', 'Shared workspace'),
  ('22222222-2222-2222-2222-222222222222', 'B private'),
  ('33333333-3333-3333-3333-333333333333', 'C private');

insert into memberships (workspace_id, user_id, role) values
  ('11111111-1111-1111-1111-111111111111', 'USER_A_UUID', 'admin'),
  ('11111111-1111-1111-1111-111111111111', 'USER_B_UUID', 'member'),
  ('22222222-2222-2222-2222-222222222222', 'USER_B_UUID', 'admin'),
  ('33333333-3333-3333-3333-333333333333', 'USER_C_UUID', 'admin');

insert into documents (workspace_id, created_by, title, body) values
  ('11111111-1111-1111-1111-111111111111', 'USER_A_UUID', 'A shared doc', 'A and
    B'),
  ('11111111-1111-1111-1111-111111111111', 'USER_B_UUID', 'B shared doc', 'A and
    B'),
  ('22222222-2222-2222-2222-222222222222', 'USER_B_UUID', 'B private doc', 'B
    only'),
  ('33333333-3333-3333-3333-333333333333', 'USER_C_UUID', 'C private doc', 'C
    only'),
  ('33333333-3333-3333-3333-333333333333', 'USER_C_UUID', 'C secrets', 'salary
    data');

insert into comments (document_id, author_id, body)
select id, created_by, 'first comment' from documents;

```

REFERENCE 3

The impersonation harness

Runs policy tests as a specific user, inside a transaction that rolls back so nothing is left behind.

```

begin;
  select set_config('request.jwt.claims',
    json_build_object('sub', 'USER_B_UUID', 'role', 'authenticated')::text, true);
  set local role authenticated;

  -- sanity check: must return USER_B_UUID, not null
  select auth.uid();

  -- now run anything as user B
  select id, title from documents;
rollback;

```

WARNING

Policy tests run outside this pattern will pass regardless of the policies, because the SQL editor's own role bypasses RLS.

REFERENCE 4

Lab 4 assertions

Run these after writing your policies, substituting your UUIDs.

```
-- R1: B cannot see C's documents
begin;
  select set_config('request.jwt.claims',
    json_build_object('sub','USER_B_UUID','role','authenticated')::text, true);
  set local role authenticated;
  select case when count(*) = 0 then 'PASS' else 'FAIL' end as r1_no_cross_read
  from documents
  where workspace_id = '33333333-3333-3333-3333-333333333333';
rollback;

-- R1b: B CAN see the two shared docs plus their own private one
begin;
  select set_config('request.jwt.claims',
    json_build_object('sub','USER_B_UUID','role','authenticated')::text, true);
  set local role authenticated;
  select case when count(*) = 3 then 'PASS' else 'FAIL' end as r1b_sees_own
  from documents;
rollback;

-- R2: B cannot insert into C's workspace (the error is the pass)
begin;
  select set_config('request.jwt.claims',
    json_build_object('sub','USER_B_UUID','role','authenticated')::text, true);
  set local role authenticated;
  insert into documents (workspace_id, created_by, title)
    values ('33333333-3333-3333-3333-333333333333','USER_B_UUID','intrusion');
rollback;

-- R3: B cannot move their own doc into C's workspace
begin;
  select set_config('request.jwt.claims',
    json_build_object('sub','USER_B_UUID','role','authenticated')::text, true);
  set local role authenticated;
  update documents set workspace_id = '33333333-3333-3333-3333-333333333333'
  where title = 'B private doc';
  select case when count(*) = 0 then 'PASS' else 'FAIL' end as r3_no_move
  from documents where title = 'B private doc'
  and workspace_id = '33333333-3333-3333-3333-333333333333';
rollback;
```

Extending this pattern to requirements 4, 5 and 6 is part of the lab, and the same pattern applies to any table you protect later.

FURTHER

If you want to go deeper

- **Postgres RLS documentation** — the authoritative source, and shorter than it looks. The `using` and `with check` section repays a second reading.
- **Supabase RLS guide** — the practical patterns, including performance notes. Policies that call a function per row can get slow, and the fixes are easier to apply before that becomes a problem.
- **OWASP: Broken Access Control** — the industry framing of this material, and the top entry on their list for several years running.

NEXT MODULE

Secrets and environments — where keys live, what must never enter a client bundle, and how to rotate one when it does. Lab 2E leads into it.